

Assignments

1. Write a function using the ternary operator to check if a number is positive, negative, or zero. E.g.: Input → 0 Output → "Zero"
Code of the program & screenshot of the output.
2. Write a function that safely accesses a nested property using optional chaining. E.g.: Input → obj = { a: { b: { c: 10 } } } → access obj.a.b.c Output → 10
Code of the program & screenshot of the output.
3. Write a program that uses short-circuiting () to assign default values. E.g.: Input → userInput = "" Output → Default Value
Code of the program & screenshot of the output.
4. Write a function that creates an object using enhanced object literals. E.g.: Input → name = "John", age = 25 Output → { name: "John", age: 25 }
Code of the program & screenshot of the output.
5. Write a function that accepts rest parameters and returns their sum. E.g.: Input → sum(1, 2, 3, 4) Output → 10
Code of the program & screenshot of the output.
6. Write a function that uses array destructuring to extract values E.g.: Input → [10, 20, 30] Output → a = 10, b = 20, c = 30
Code of the program & screenshot of the output.
7. Write a function that uses object destructuring to extract nested properties. E.g.: Input → { user: { name: "Alice", age: 22 } } Output → name = "Alice", age = 22
Code of the program & screenshot of the output.
8. Create two files: one exports a function, the other imports it using ES6 Modules. E.g.: Export add() from one file and import in another. Output → Correct execution of add(2, 3) → 5
Code of the program & screenshot of the output.
9. Write a function that takes another function as an argument (callback) and executes

it. E.g.: Input → greet() => "Hello" Output → "Hello"
Code of the program & screenshot of the output.
10. Write a program that fetches data using async/await and logs the result. E.g.: Use fetch('https://jsonplaceholder.typicode.com/todos/1') Output → Fetched todo item
Code of the program & screenshot of the output.
11. Write a generator function to produce first n even numbers. E.g.: Input → n = 4 Output → 0, 2, 4, 6
Code of the program & screenshot of the output.
12. Create a class called Person with name and age, and a method greet() that returns "Hello [name]". E.g.: Input → Person("Alice", 25) Output → "Hello Alice"
Code of the program & screenshot of the output.
13. Create a subclass Student that extends Person and adds grade. E.g.: Input → Student("Bob", 20, "A") Output → Hello Bob, Grade: A
Code of the program & screenshot of the output.
14. Write a class using encapsulation where age can be accessed only through a getter. E.g.: Input → person.getAge() Output → 25
Code of the program & screenshot of the output.
15. Implement polymorphism using method overriding in a class hierarchy. E.g.: Animal → speak(), Dog → speak() Output → Dog says woof!
Code of the program & screenshot of the output.
16. Write a program that filters an array of users using filter() and destructuring. E.g.: Input → users with age > 18 Output → Filtered list of adults
Code of the program & screenshot of the output.
17. Write a program that uses async/await with try...catch for error handling. E.g.: Input → Bad URL Output → Logs error properly

Code of the program & screenshot of the output.
18. Write a generator function to yield values from 1 to n. E.g.: Input → n = 5 Output → 1, 2, 3, 4, 5
Code of the program & screenshot of the output.
19. Create a class that accepts rest parameters in its constructor and stores all items. E.g.: Input → new Basket("apple", "banana") Output → ["apple", "banana"]
Code of the program & screenshot of the output.
20. Write a class method using optional chaining to check if an optional property exists. E.g.: Input → this.address?.city Output → "Chennai" or undefined
Code of the program & screenshot of the output.